Learning objectives for the updated module

- Choose an appropriate execution model: sequential, vectorised, process-based, scheduler-based, or GPU.
- Use the `future` ecosystem for portable parallel code.
- Run reproducible parallel simulations.
- Scale the same computation from a workstation to a Slurm cluster.
- Recognise when faster I/O or an analytical database is better than adding workers.

A modern performance workflow

1. Measure the real bottleneck: CPU, memory, I/O, or communication.
2. Improve the algorithm and data representation.
3. Use vectorised or compiled routines.
4. Parallelise only sufficiently large, independent tasks.
5. Scale across nodes only when one node is insufficient.
6. Verify correctness, reproducibility, memory use, and speedup.

Parallel execution is not automatically faster. Startup, data transfer, serialisation, and load imbalance can dominate short tasks.

How many workers should I use?

Use the resources available to the current process

```
library(parallelly)

availableCores()        # respects many scheduler/co
availableWorkers()      # workers visible to the cur

detectCores()           # hardware/logical CPUs, not
```

- Logical processors are not necessarily equally fast on hybrid CPUs.
- On an HPC system, use the CPUs granted by the scheduler, not the whole node.
- Leave capacity for the operating system on an interactive workstation.

Processes, threads, and oversubscription

- **R workers** are usually separate processes with separate memory.
- BLAS, OpenMP, `data.table`, and compiled libraries may create additional threads.
- Eight R workers with eight BLAS threads each can create up to 64 runnable threads.
- Oversubscription often reduces performance and makes timings unstable.

Practical rule: when using many R workers, initially set threaded numerical libraries to one thread per worker, benchmark, and then tune.

Benchmark elapsed time and memory

Prefer representative workloads

```
library(bench)

x <- runif(1e6)
bench::mark(
  loop = {
    ans <- 0
    for (i in seq_along(x)) ans <- ans + x[[i]]
    ans
  },
  vectorised = sum(x),
  check = TRUE
)
```

- Compare results, not just timings.
- Report elapsed time, memory allocation, workload size, worker count, and hardware.

Why teach the `future` ecosystem?

- It separates **what** should be evaluated from **where and how** it runs.
- The computational code can remain stable while the execution plan changes.
- Common front ends include `future.apply`, `furrr`, and `doFuture`.
- Backends cover local processes, remote workers, and scheduler-managed HPC jobs.

Keep `parallel` and `foreach` in the course as foundations and legacy interfaces, but use `future` as the main portable workflow.

From sequential to parallel with one change

Portable parallel map

```
library(future)
library(future.apply)

f <- function(i) {
  x <- rnorm(2e6)
  c(task = i, statistic = mean(exp(x)))
}

plan(sequential)
a <- future_lapply(1:8, f, future.seed = TRUE)

plan(multisession, workers = 4)
b <- future_lapply(1:8, f, future.seed = TRUE)

identical(a, b)
plan(sequential)
```

Parallel tidy workflows with `furrr`

The `purrr` interface with futures

```
library(future)
library(furrr)

plan(multisession, workers = 4)

results <- future_map(
  1:20,
  ~ mean(rnorm(1e6, mean = .x)),
  .options = furrr_options(seed = 2026)
)

plan(sequential)
```

Use this interface when students already know `purrr::map()`.

Keep existing `foreach` code with `doFuture`

Modern backend for a familiar interface

```
library(future)
library(doFuture)
library(foreach)

registerDoFuture()
plan(multisession, workers = 4)

ans <- foreach(i = 1:20,
               .combine = c,
               .options.future = list(seed = TRUE)) %dopar% {
  mean(rnorm(1e6, mean = i))
}

plan(sequential)
```

This provides a migration path rather than requiring an immediate rewrite of all `foreach` material

Choosing a local future plan

- `sequential`: debugging, teaching, and a reproducible baseline.
- `multisession`: separate R processes; portable across Windows, macOS, and Linux.
- `multicore`: forked processes; available on supported POSIX systems, not Windows.
- Explicit cluster backends: use when workers are remote or require custom startup.

For a mixed classroom, `multisession` is the safest default. Always return to `plan(sequential)` after an exercise.

Reproducible parallel random numbers

Do not rely only on `set.seed()` before parallel mapping

```r
library(future.apply)

plan(multisession, workers = 4)
set.seed(2026)

z1 <- future_lapply(1:10, function(i) rnorm(3),
                    future.seed = TRUE)

set.seed(2026)
z2 <- future_lapply(1:10, function(i) rnorm(3),
                    future.seed = TRUE)

identical(z1, z2)
plan(sequential)
```

Reproducibility must hold when the worker count or backend

Common reasons for disappointing speedup

- Tasks are too short compared with worker startup and communication.
- Large global objects are copied or serialised for every task.
- Workers compete for memory bandwidth or disk I/O.
- Work is unevenly distributed among workers.
- Nested parallelism oversubscribes CPUs.
- A sequential fraction limits total speedup (Amdahl's law).

Measure serial and parallel versions using the same inputs and verify that they return equivalent results.

Chunking: amortise scheduling overhead

Prefer fewer substantial tasks to millions of tiny tasks

```
library(future.apply)

plan(multisession, workers = 4)

ids <- split(1:10000, cut(1:10000, 40, labels = FAL
ans <- future_lapply(ids, function(block) {
  vapply(block, expensive_function, numeric(1))
})
ans <- unlist(ans, use.names = FALSE)
```

The best chunk size depends on task duration, variability, memory use, and backend latency.

A scheduler changes resource ownership

- The login node is for editing, compiling, and submitting jobs, not heavy computation.
- Slurm grants CPUs, memory, accelerators, nodes, and wall time to a job.
- R should discover and use only the resources allocated to that job.
- Multi-node execution adds network communication and distributed failure modes.

A minimal Slurm batch job

`run_analysis.sbatch`

```
#!/bin/bash
#SBATCH --job-name=parallel-r
#SBATCH --nodes=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --time=00:30:00
#SBATCH --output=logs/%x_%j.out

module load R
Rscript analysis.R
```

Submit with `sbatch run_analysis.sbatch`; inspect the queue with `squeue -u $USER`. Adapt partition, account, module name, memory, and time to the target system.

Use Slurm through `future.batchtools`

Change the plan, not the analytical function

```
library(future)
library(future.apply)
library(future.batchtools)

plan(
  future.batchtools::batchtools_slurm,
  resources = list(
    time = "00:20:00",
    mem = "4G",
    ncpus = 1
  )
)

ans <- future_lapply(files, analyse_file,
                     future.seed = TRUE)
```

Single node or multiple nodes?

- Start with one node: lower latency, simpler debugging, and usually faster data access.
- Use multiple nodes when memory, throughput, or turnaround requirements exceed one node.
- Independent simulations and file-level processing scale more easily than tightly coupled algorithms.
- For communication-heavy numerical algorithms, specialised MPI or distributed libraries may be more appropriate.

Sometimes the bottleneck is data, not R code

- CSV parsing, repeated scans, and object copying can dominate the computation.
- Columnar formats such as Parquet reduce unnecessary I/O for analytical workloads.
- Apache Arrow supports columnar and dataset-oriented workflows.
- DuckDB can execute analytical SQL directly over local files and tabular data.

Before adding workers, reduce the amount of data read, copied, and returned.

Query Parquet data without loading everything

Arrow dataset with lazy filtering and aggregation

```
library(arrow)
library(dplyr)

sales <- open_dataset("data/sales", format = "parqu

summary <- sales |>
  filter(year == 2026) |>
  group_by(region) |>
  summarise(revenue = sum(amount, na.rm = TRUE)) |>
  collect()
```

Only call `collect()` when the reduced result is needed as an in-memory R object.

Use DuckDB for analytical queries

Push computation into an analytical database

```
library(DBI)
library(duckdb)

con <- dbConnect(duckdb())
result <- dbGetQuery(con, "
--SELECT region, AVG(value) AS mean_value
--FROM read_parquet('data/*.parquet')
--GROUP BY region
")
dbDisconnect(con, shutdown = TRUE)
```

Compare this approach with manually splitting files among R workers.

Four distinct acceleration strategies

- **Vectorisation**: use optimised compiled routines already available to R.
- **Process parallelism**: distribute independent R tasks among workers.
- **Compiled extensions**: move a hot loop to C++ with `Rcpp`.
- **Accelerators**: use GPU-aware libraries for suitable dense, batched, or model-training workloads.

These strategies can be combined, but nested parallelism and data transfers must be controlled.

Extend the Mandelbrot example with `RcppParallel`

- Keep the existing three implementations: R, R with a C++ kernel, and complete C++.
- Add a fourth implementation using `RcppParallel::parallelFor`.
- Partition grid indices among threads and write each output element once.
- Compare elapsed time and scaling for 1, 2, 4, and 8 threads.
- Discuss grain size, memory bandwidth, thread safety, and reproducibility.

GPU tensors from R with `torch`

Use an accelerator-aware library rather than explicit GPU loops

```
library(torch)

device <- if (cuda_is_available()) torch_device("cu

x <- torch_randn(c(4000, 4000), device = device)
y <- torch_randn(c(4000, 4000), device = device)
z <- torch_matmul(x, y)

z_cpu <- z$to(device = "cpu")
```

GPU speedup depends on operation size, supported kernels, device memory, and host–device transfer costs.

Capstone: parallel model evaluation

1. Create a fixed resampling or hyperparameter grid.
2. Implement one deterministic function that evaluates a single grid row.
3. Run it sequentially and verify results.
4. Run locally with `future_map()` or `future_lapply()`.
5. Run the same function through a Slurm backend.
6. Report elapsed time, speedup, efficiency, memory, and failed tasks.

Capstone skeleton

Backend-independent worker function

```
library(future)
library(furrr)

grid <- expand.grid(
  penalty = 10^seq(-4, 1, length.out = 20),
  split = 1:10
)

evaluate_one <- function(i) {
  row <- grid[i, ]
  # fit on the training split; score on the test sp
  data.frame(i = i, penalty = row$penalty,
             split = row$split, score = NA_real_)
}

plan(multisession, workers = 4)
```

Updated summary

- Optimise the algorithm and data movement before parallelising.
- Use `parallel` to explain processes, PSOCK, and forked execution.
- Use `future` as the primary portable parallel interface.
- Treat reproducible random numbers and worker cleanup as core correctness topics.
- Use Slurm-aware backends for HPC scaling.
- Consider Arrow, DuckDB, compiled code, or GPUs when they match the bottleneck.

References and documentation

- Futureverse: `https://www.futureverse.org/`
- `future.batchtools`: `https://future.batchtools.futureverse.org/`
- `parallelly`: `https://parallelly.futureverse.org/`
- Arrow R documentation: `https://arrow.apache.org/docs/r/`
- DuckDB R client: `https://duckdb.org/docs/stable/clients/r`
- `RcppParallel`: `https://rcppcore.github.io/RcppParallel/`
- torch for R: `https://torch.mlverse.org/`
- Slurm documentation: `https://slurm.schedmd.com/documentation.html`